project-corebank 작업 정리

미니 코어뱅킹 플랫폼 (금융권 포트폴리오 사이드 프로젝트) · 기준일 2026-11-03 · 작업 기간 2026-11-02 ~ 11-03

계좌 개설·조회, 입금, 출금, 계좌 간 이체, 멱등성 키까지 완료했습니다. 모든 거래는 복식부기 원장에 기록되고, 실제 Oracle 위에서 단위·통합 테스트와 Postman 회귀 테스트, DB 원장 대사를 모두 통과했습니다.

5개	101 / 101	200 / 200	0건
완성한 API	단위·통합 테스트 통과	Postman 검증 통과	DB 원장 대사 불일치

1. 진행 이력

날짜	작업	내용
11-02	프로젝트 설정	Java 25 LTS, Spring Boot 4.1.2, Gradle, JPA, Oracle(Docker/Colima), Flyway, Testcontainers 구성. 설계 문서를 README로 정리
11-02	계좌 개설·조회	계좌번호 생성(은행 코드 110 + 10자리 일련번호), 입력 검증, 공통 에러 형식, Lombok `@Getter` 리팩토링
11-02	입금	비관적 락, 복식부기 원장(거래·분개 테이블), 소수 금액 거부, 잔액 overflow 방지
11-03	출금	잔액 부족 검사(422), 원장 분개를 입금과 반대 방향으로 기록, 동시 출금 시 마이너스 잔액 방지
11-03	API 전체 점검	Postman 회귀 컬렉션 작성. 415/405/404 응답을 공통 에러 형식으로 수정
11-03	이체	계좌 잠금 순서를 고정해 데드락 방지, 이체 가계정, 반쪽 이체 방지(전체 롤백)
11-03	멱등성 키	입금·출금·이체에 `Idempotency-Key` 헤더 필수, 재전송 시 한 번만 처리하고 처음 결과를 돌려줌

2. 기술 스택과 구조

구분	사용 기술
언어·프레임워크	Java 25, Spring Boot 4.1.2 (WebMVC, Data JPA, Validation)
빌드	Gradle (toolchain Java 25)
DB	Oracle Free 23 (Docker, Colima)
스키마 관리	Flyway (V1 ~ V5)
테스트	JUnit 5, MockMvc, Testcontainers(실제 Oracle), Postman CLI

패키지	역할
`account`	계좌 엔티티와 업무 규칙, 입출금 서비스와 API
`transfer`	이체 서비스와 API, 이체 기록
`ledger`	복식부기 원장(거래, 분개) 기록과 조회
`idempotency`	멱등성 키 검증, 기록, 재전송 응답
`common`	공통 에러 응답, 예외 처리

소스 파일: 운영 코드 43개, 테스트 15개

3. API 명세

기능	API	요청 본문	성공 응답
계좌 개설	`POST /accounts`	`ownerName`	201, Location 헤더, 계좌 정보 (잔액 0, ACTIVE)

계좌 조회	`GET /accounts/{id}`	없음	200, 계좌 정보
입금 *	`POST /accounts/{id}/deposits`	`amount`	201, 거래 번호, 거래 후 잔액
출금 *	`POST /accounts/{id}/withdrawals`	`amount`	201, 거래 번호, 거래 후 잔액
이체 *	`POST /transfers`	`fromAccountId`, `toAccountId`, `amount`	201, Location 헤더, 이체 번호, 두 계좌의 이체 후 잔액

* `Idempotency-Key` 헤더 필수. 같은 키로 재전송하면 같은 201 본문과 `Idempotent-Replayed: true` 헤더를 돌려줌. 금액은 1원 이상의 정수만 허용(소수·문자 거부).

에러 응답 (모두 `{"code", "message"}` 공통 형식)

상태	코드	발생 조건
400	`INVALID_REQUEST`	입력 검증 실패, 깨진 JSON, 숫자가 아닌 경로 값
400	`SAME_ACCOUNT_TRANSFER`	보내는 계좌와 받는 계좌가 같음
400	`IDEMPOTENCY_KEY_MISSING`	입금·출금·이체에 키 헤더 없음
400	`INVALID_IDEMPOTENCY_KEY`	키 형식 오류 (영문·숫자·`-`·`_`, 1~100자만 허용)
404	`ACCOUNT_NOT_FOUND`	없는 계좌
404	`RESOURCE_NOT_FOUND`	없는 경로
405	`METHOD_NOT_ALLOWED`	지원하지 않는 HTTP 메서드 (Allow 헤더 포함)
409	`ACCOUNT_CLOSED`	해지된 계좌로 거래
415	`UNSUPPORTED_MEDIA_TYPE`	JSON이 아닌 Content-Type (Accept 헤더 포함)
422	`INSUFFICIENT_BALANCE`	잔액 부족
422	`BALANCE_LIMIT_EXCEEDED`	잔액이 long 범위를 넘음
422	`IDEMPOTENCY_KEY_REUSED`	이미 쓴 키로 다른 내용의 요청

4. 핵심 설계

4-1. 복식부기 원장

잔액을 바꾸는 모든 거래는 원장 거래 1건과 분개 2줄(차변·대변)로 남고, 차변 합과 대변 합은 항상 같습니다. 원장은 추가만 하고 수정하지 않습니다.

거래	차변	대변
입금	현금 (은행 자산 증가)	고객 예금 (은행 부채 증가)
출금	고객 예금 (부채 감소)	현금 (자산 감소)
이체 · 보내는 쪽	고객 예금 (보내는 계좌)	이체 가계정
이체 · 받는 쪽	이체 가계정	고객 예금 (받는 계좌)

이체는 계좌별 원장 거래 2건(`TRANSFER_OUT` , `TRANSFER_IN`)을 같은 이체 번호로 묶어서, 양쪽 계좌 모두 거래 후 잔액이 남습니다. 이체 가계정은 이체가 끝나면 합계가 0으로 돌아오므로, 0이 아니면 반쪽만 기록된 이체가 있다는 뜻입니다.

4-2. 동시성 제어: 비관적 락

거래 전에 `SELECT ... FOR UPDATE` 로 계좌를 잠가, 같은 계좌의 거래를 한 줄로 세웁니다.

- **락을 뺀 실험 (입금):** 1,000원씩 동시 입금 20건 → 20,000원이어야 할 잔액이 4,000원이 됨 (서로의 잔액을 덮어씀)
- **락을 뺀 실험 (출금):** 잔액 10,000원 계좌에서 3,000원 출금 10건 → 3건만 성공해야 하는데 10건 모두 성공. DB의 `balance >= 0` 제약으로도 막지 못함

4-3. 데드락 방지: 잠금 순서 고정

이체는 두 계좌를 잠그므로, 보내는 계좌부터 잠그면 A→B와 B→A 이체가 서로를 기다리며 멈춥니다. 이체 방향과 관계없이 **id가 작은 계좌부터** 잠그도록 고정했습니다.

- **순서를 바꾼 실험:** 반대 방향 이체 40건 동시 실행 → Oracle이 교착 상태를 감지해 일부를 `ORA-00060` 으로 실패시킴
- **고정한 뒤:** 실제 서버에 반대 방향 이체 60건을 동시에 보내 모두 성공, 두 계좌 잔액 보존

4-4. 트랜잭션: 전부 아니면 전무

계좌 잠금, 잔액 변경, 원장 기록, 멱등성 키 기록을 한 트랜잭션으로 처리합니다. 받는 계좌가 한도에 걸려 이체가 실패하면, 이미 차감한 보내는 계좌의 잔액도 함께 되돌아갑니다. 분개 저장을 일부러 실패시키는 경우도 테스트로 확인했습니다.

4-5. 멱등성 키

- 성공한 요청은 키, 요청 내용 요약(예: `DEPOSIT|account=1|amount=1000`), 만들어진 거래 번호를 함께 저장합니다.
- **계좌를 잠근 뒤에 키를 확인합니다.** 같은 요청이 동시에 오면 두 번째는 잠금에서 기다렸다가, 첫 번째가 커밋한 결과를 받습니다.
  - 키 확인을 잠금 앞으로 옮긴 실험: 돈은 유니크 제약 덕분에 두 번 들어가지 않았지만, 정상적인 재전송 요청이 "키 재사용(422)" 오류를 받음
- 재전송에는 "지금 잔액"이 아니라 처음 처리했을 때의 응답을 그대로 돌려줍니다.
- DB 유니크 제약이 마지막 방어선입니다. 같은 키로 서로 다른 계좌에 동시에 요청하면 하나만 남고 나머지는 거부됩니다.

5. DB 스키마 (Flyway)

버전	내용
`V1` create_account	`account` 테이블. 잔액 `>= 0` , 상태 값 CHECK 제약, 계좌번호 유니크

`V2` create_ledger	`ledger_transaction`(거래), `ledger_entry`(분개), 외래키 인덱스
`V3` allow_withdrawal	거래 종류에 `WITHDRAWAL` 추가
`V4` create_transfer	`transfer` 테이블, 원장 거래에 이체 번호 연결, 거래 종류 2개 추가, 이체 가계정 허용
`V5` create_idempotency_record	`idempotency_record` 테이블, 키 유니크 제약

6. 검증 결과

6-1. 단위·통합 테스트: 101개 모두 통과

테스트 클래스	종류	개수
AccountTest	단위 (계좌 업무 규칙)	9
AccountControllerTest	웹 계층 (계좌·입출금·HTTP 규약)	24
TransferControllerTest	웹 계층 (이체)	14
IdempotencyControllerTest	웹 계층 (멱등성 키 헤더)	13
AccountIntegrationTest	통합 (실제 Oracle)	3
DepositIntegrationTest	통합	5
WithdrawalIntegrationTest	통합	7
TransferIntegrationTest	통합 (데드락, 반쪽 이체 롤백 포함)	11
IdempotencyIntegrationTest	통합 (동시 재전송 포함)	11
DepositRollbackIntegrationTest, TransferRollbackIntegrationTest	롤백 검증	3
ProjectCorebankApplicationTests	애플리케이션 기동	1
합계		**101**

6-2. Postman 회귀 컬렉션: 요청 81개, 검증 200개 모두 통과

폴더	요청	주요 확인 내용
1. 계좌 개설/조회	11	Location 헤더, 계좌번호 중복 없음, 이름 100자 허용·101자 거부, 잔액·상태 조작 무시
2. 입금	13	잔액 누적, 잘못된 금액 5종 거부, 없는 계좌, long 한도 초과
3. 출금	12	잔액 부족, 잔액 전부 출금, 0원에서 1원 출금 거부
4. 이체	24	양쪽 잔액, 반대 방향 이체, 같은 계좌, 입금 쪽 실패 시 롤백
5. 멱등성 키	18	재전송 시 같은 결과, 키 재사용 거부, 실패한 키 재시도
6. HTTP 규약	3	415 / 405 / 404 공통 에러 형식과 헤더

6-3. 실제 서버에 동시 요청 (curl 병렬 호출)

시나리오	결과
잔액 10,000원에서 3,000원 출금 10건 동시	**3건 성공, 7건 잔액 부족, 잔액 1,000원**
1,000원 입금 30건 동시	**30건 성공, 잔액 30,000원**
A↔B 반대 방향 이체 60건 동시	**60건 성공, 데드락 없음, 두 계좌 잔액 보존**
같은 멱등성 키로 10,000원 입금 20건 동시	**20건 모두 201, 실제 입금은 1번 (잔액 10,000원)**

6-4. DB 원장 대사 (개발 DB): 불일치 모두 0건

- 전체 원장의 차변 합과 대변 합이 같음
- 이체 가계정 합계가 0
- 모든 계좌의 잔액이 원장 분개 합계와 같음
- 모든 거래가 분개 2줄이고 차변과 대변이 맞음

- 모든 이체가 보내는 쪽 거래 1건과 받는 쪽 거래 1건으로 기록됨
- 계좌별 마지막 거래의 거래 후 잔액이 현재 잔액과 같음
- 모든 멱등성 기록이 실제 거래를 가리킴

7. 남은 한계와 다음 단계

남은 한계	내용
계좌 해지 API 없음	해지 계좌(409)는 테스트에서 DB 상태를 직접 바꿔서만 확인함
원장 불변성	코드로는 수정을 막았지만 DB에 직접 SQL을 실행하면 수정 가능
인증 없음	이체 응답에 받는 계좌의 잔액이 노출됨. 멱등성 키도 사용자별로 나뉘지 않음
멱등성 키 만료 없음	키가 영구 보관됨 (실제 서비스는 보통 24시간)
500 오류 형식	예상하지 못한 서버 오류는 아직 스프링 기본 형식으로 응답함
시험 데이터	개발 DB에 시험용 계좌와 거래가 남아 있음

다음 단계 (README 계획 기준): 거래내역 조회 → 계좌 해지 → 일 마감 대사 배치(Spring Batch) → 인증 → 고객·관리자 화면

8. 실행 방법

```
# 1. Docker 실행 (PC를 재부팅한 뒤에는 필요)
colima start
docker compose up -d --wait        # 개발용 Oracle

# 2. 테스트 (Testcontainers 가 테스트용 Oracle 을 따로 띄움)
./gradlew test

# 3. 서버 실행 → http://localhost:8080
./gradlew bootRun

# 4. API 회귀 테스트 (서버 실행 중일 때)
postman collection run postman/corebank-api.postman_collection.json
```

> Postman에서 입금·출금·이체를 직접 호출할 때는 Headers에 `Idempotency-Key: {{$guid}}` 를 추가해야 합니다.